

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КАРПАТСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ВАСИЛЯ
СТЕФАНИКА

Факультет математики та інформатики
Кафедра комп'ютерних наук та інформаційних систем

ЛАБОРАТОРНА РОБОТА № 1

з дисципліни «Програмування C#»

Тема: «Візуальне моделювання фігур Ліссажу»

Виконав:

студент групи КН-21

Соколов Олександр Сергійович

Прийняв:

к.т.н,доц. Ровінський В.А.

Івано-Франківськ – 2026

1. Тема і мета роботи

Тема: Візуальне моделювання фігур Ліссажу.

Мета роботи: дослідити предметну область гармонічних коливань і фігур Ліссажу, побудувати їх математичну модель та створити інтерактивний WPF-застосунок мовою C# із використанням XAML, Binding, команд та підходу MVVM.

2. Короткий опис предметної області та математичної моделі

Фігури Ліссажу утворюються в результаті накладання двох взаємно перпендикулярних гармонічних коливань. Їх форма залежить від амплітуд, частот та фазових зсувів двох коливань. Такі фігури можуть використовуватися для порівняння частот і фаз сигналів.

У програмі використано параметричні рівняння:

$$x(t) = A_x \cdot \sin(2\pi f_x \cdot t + \varphi_x)$$

$$y(t) = A_y \cdot \sin(2\pi f_y \cdot t + \varphi_y)$$

Фази користувач задає у градусах. Перед використанням функції Math.Sin() вони переводяться у радіани:

$$\varphi_{\text{rad}} = \varphi_{\text{deg}} \cdot \pi / 180$$

Період коливання визначається формулою $T = 1/f$. Для чисельної побудови кривої використовується дискретизація часу. У програмі крок Δt перевіряється так, щоб на найшвидший період припадало не менше 20 точок.

3. Структура програми

Програму побудовано за принципами WPF та мінімальної архітектури MVVM. Логіка обчислень відокремлена від графічного інтерфейсу.

- **PointD:** модель однієї математичної точки з координатами X та Y .
- **LissajousParameters:** зберігає параметри моделі: A_x , A_y , f_x , f_y , ϕ_x , ϕ_y та Δt .
- **LissajousModel:** виконує перевірку параметрів і розраховує набір точок фігури Ліссажу.
- **MainViewModel:** зберігає стан програми, реалізує `INotifyPropertyChanged`, перевіряє введення та передає параметри моделі.
- **RelayCommand:** реалізує `ICommand` для прив'язування кнопок до дій `ViewModel`.
- **MainWindow.xaml:** містить інтерфейс: `TextBox`, `Slider`, `Button`, `Canvas` і `Polyline`.
- **MainWindow.xaml.cs:** містить лише запуск вікна та не містить математичних обчислень.

4. Ключові алгоритми та фрагменти коду

4.1. Розрахунок точок фігури

```
double phaseX = parameters.PhaseXDegrees * Math.PI / 180.0;
double phaseY = parameters.PhaseYDegrees * Math.PI / 180.0;

double minFrequency =
    Math.Min(parameters.FrequencyX, parameters.FrequencyY);

double duration = 5.0 / minFrequency;

int pointCount =
    (int)Math.Ceiling(duration / parameters.TimeStep) + 1;

var points = new List<PointD>(pointCount);

for (int i = 0; i < pointCount; i++)
{
    double t = i * parameters.TimeStep;
```

```

double x = parameters.AmplitudeX *
    Math.Sin(2 * Math.PI *
        parameters.FrequencyX * t + phaseX);

double y = parameters.AmplitudeY *
    Math.Sin(2 * Math.PI *
        parameters.FrequencyY * t + phaseY);

points.Add(new PointD(x, y));
}

```

Цикл автоматично формує колекцію точок. Тому точки не задаються вручну, а кількість точок залежить від тривалості моделювання та кроку часу.

4.2. Перевірка вхідних даних

```

if (parameters.AmplitudeX <= 0 ||
    parameters.AmplitudeY <= 0)
{
    throw new ArgumentException(
        "Амплітуди Ax та Ay повинні бути додатними.");
}

if (parameters.FrequencyX <= 0 ||
    parameters.FrequencyY <= 0)
{
    throw new ArgumentException(
        "Частоти fx та fy повинні бути додатними.");
}

double maxFrequency =
    Math.Max(parameters.FrequencyX,
        parameters.FrequencyY);

double maximumAllowedStep =
    1.0 / (20.0 * maxFrequency);

if (parameters.TimeStep > maximumAllowedStep)
{
    throw new ArgumentException(
        "Крок часу надто великий.");
}

```

4.3. Побудова графіка з однаковим масштабом

Після розрахунку математичні координати переводяться у координати Canvas. Для X та Y використовується один і той самий коефіцієнт масштабу. Це необхідно, щоб коло при однакових амплітудах не перетворювалося на еліпс через різний масштаб осей.

```
double maxAmplitude =  
    Math.Max(parameters.AmplitudeX,  
              parameters.AmplitudeY);  
  
double scale =  
    availableSize / (2.0 * maxAmplitude);  
  
double screenX =  
    centerX + point.X * scale;  
  
double screenY =  
    centerY - point.Y * scale;
```

5. Контрольні приклади та перевірка правильності

Приклад 1 — пряма лінія

$A_x = 1$, $A_y = 1$, $f_x = 1$, $f_y = 1$, $\varphi_x = 0^\circ$, $\varphi_y = 0^\circ$. Оскільки частоти, амплітуди та фази однакові, отримуємо $y = x$ — діагональну пряму.

Фігури Ліссажу

$$x(t) = A_x \sin(2\pi f_x t + \phi_x)$$
$$y(t) = A_y \sin(2\pi f_y t + \phi_y)$$

Амплітуда A_x (> 0):

Амплітуда A_y (> 0):

Частота f_x , Гц (> 0):

Частота f_y , Гц (> 0):

Фаза ϕ_x , градуси:

Фаза ϕ_y , градуси:

-180° 0° 180°

Крок часу Δt , с (> 0):

Побудувати

Типові співвідношення частот:

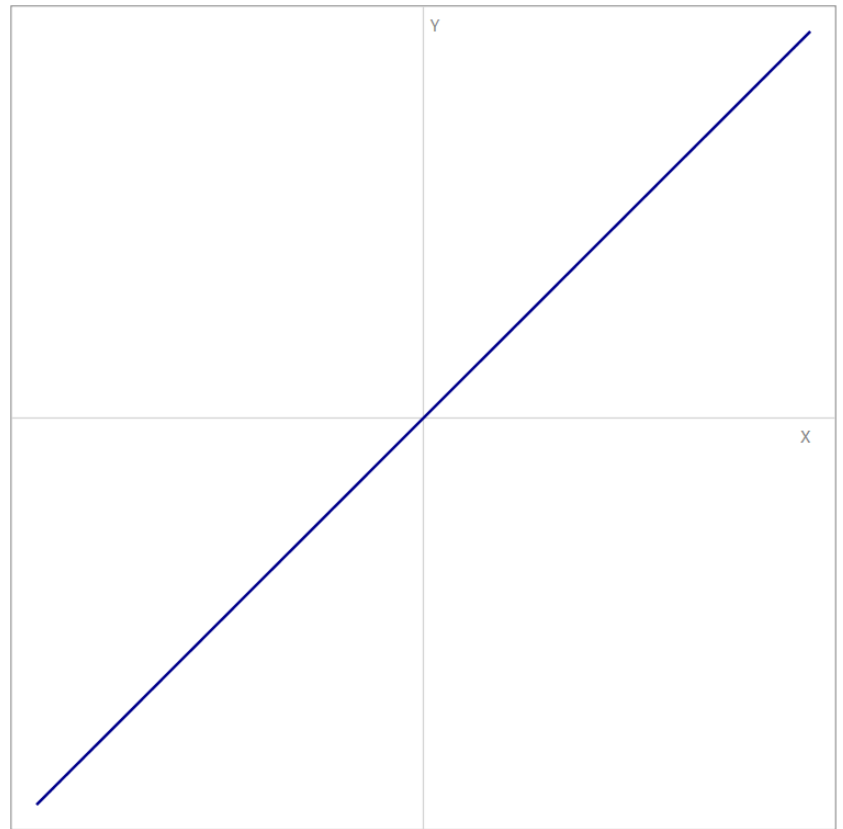
1 : 1

1 : 2

2 : 3

3 : 4

Побудовано успішно. Кількість точок: 2501.



Приклад 2 — коло

$A_x = 1$, $A_y = 1$, $f_x = 1$, $f_y = 1$, $\phi_x = 0^\circ$, $\phi_y = 90^\circ$. За однакових амплітуд і фазового зсуву 90° формується коло.

Фігури Ліссажу

$$x(t) = A_x \sin(2\pi f_x \cdot t + \varphi_x)$$

$$y(t) = A_y \sin(2\pi f_y \cdot t + \varphi_y)$$

Амплітуда A_x (> 0):

Амплітуда A_y (> 0):

Частота f_x , Гц (> 0):

Частота f_y , Гц (> 0):

Фаза φ_x , градуси:

Фаза φ_y , градуси:

-180° 0° 180°

Крок часу Δt , с (> 0):

Побудувати

Типові співвідношення частот:

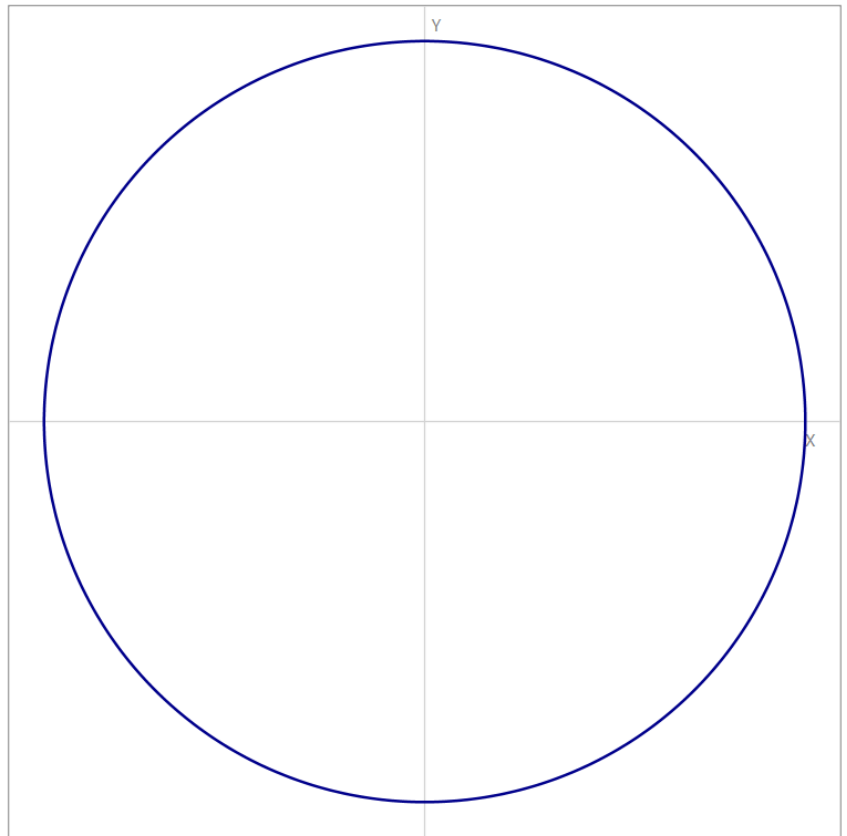
1 : 1

1 : 2

2 : 3

3 : 4

Побудовано успішно. Кількість точок: 2501.



Приклад 3 — співвідношення 1:2

$f_x = 1$, $f_y = 2$. Фігура має складнішу форму та демонструє вплив співвідношення частот.

Фігури Ліссажу

$$x(t) = A_x \cdot \sin(2\pi f_x \cdot t + \varphi_x)$$

$$y(t) = A_y \cdot \sin(2\pi f_y \cdot t + \varphi_y)$$

Амплітуда A_x (> 0):

Амплітуда A_y (> 0):

Частота f_x , Гц (> 0):

Частота f_y , Гц (> 0):

Фаза φ_x , градуси:

Фаза φ_y , градуси:

-180°

0°

180°

Крок часу Δt , с (> 0):

Побудувати

Типові співвідношення частот:

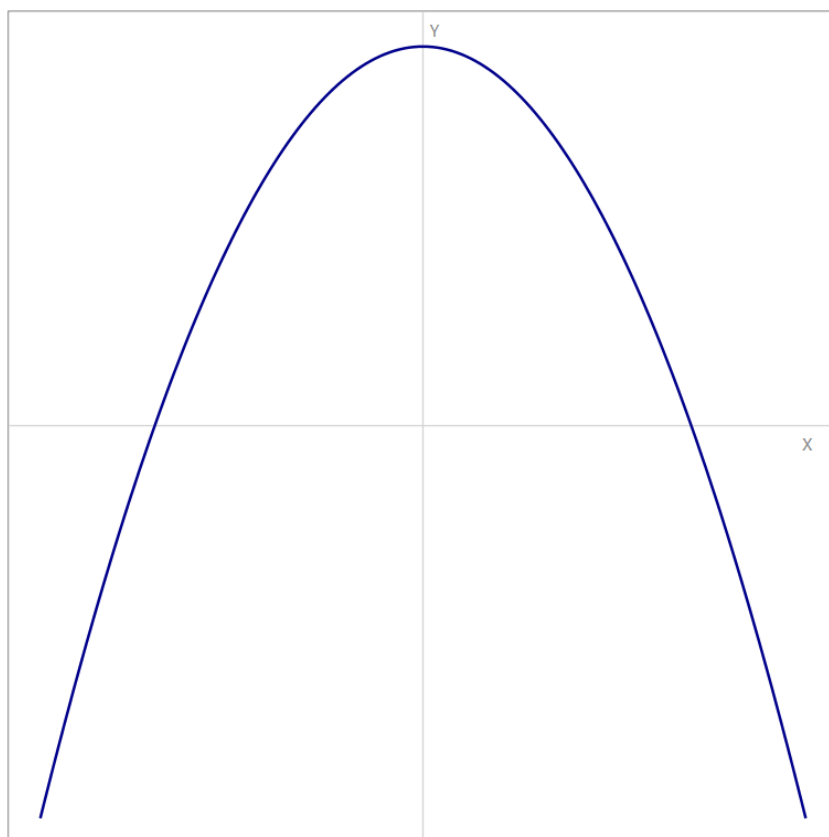
1 : 1

1 : 2

2 : 3

3 : 4

Побудовано успішно. Кількість точок: 2501.



Приклад 4 — співвідношення 2:3

$f_x = 2$, $f_y = 3$. Кількість і симетрія петель змінюються відповідно до співвідношення частот.

Фігури Ліссажу

$$x(t) = A_x \sin(2\pi f_x t + \varphi_x)$$

$$y(t) = A_y \sin(2\pi f_y t + \varphi_y)$$

Амплітуда A_x (> 0):

Амплітуда A_y (> 0):

Частота f_x , Гц (> 0):

Частота f_y , Гц (> 0):

Фаза φ_x , градуси:

Фаза φ_y , градуси:



-180° 0° 180°

Крок часу Δt , с (> 0):

Побудувати

Типові співвідношення частот:

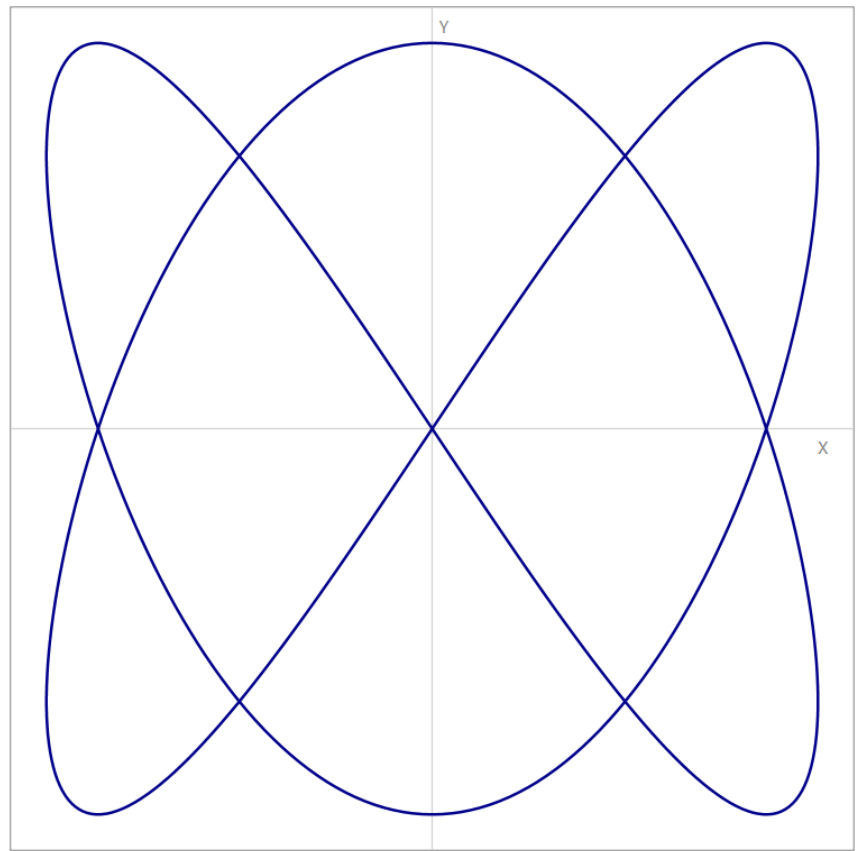
1 : 1

1 : 2

2 : 3

3 : 4

Побудовано успішно. Кількість точок: 1251.



Приклад 5 — співвідношення 3:4

$f_x = 3$, $f_y = 4$. Отримується складна фігура Ліссажу з більшим числом петель.

Фігури Ліссажу

$$x(t) = A_x \sin(2\pi f_x t + \phi_x)$$

$$y(t) = A_y \sin(2\pi f_y t + \phi_y)$$

Амплітуда A_x (> 0):

Амплітуда A_y (> 0):

Частота f_x , Гц (> 0):

Частота f_y , Гц (> 0):

Фаза ϕ_x , градуси:

Фаза ϕ_y , градуси:

-180° 0° 180°

Крок часу Δt , с (> 0):

Побудувати

Типові співвідношення частот:

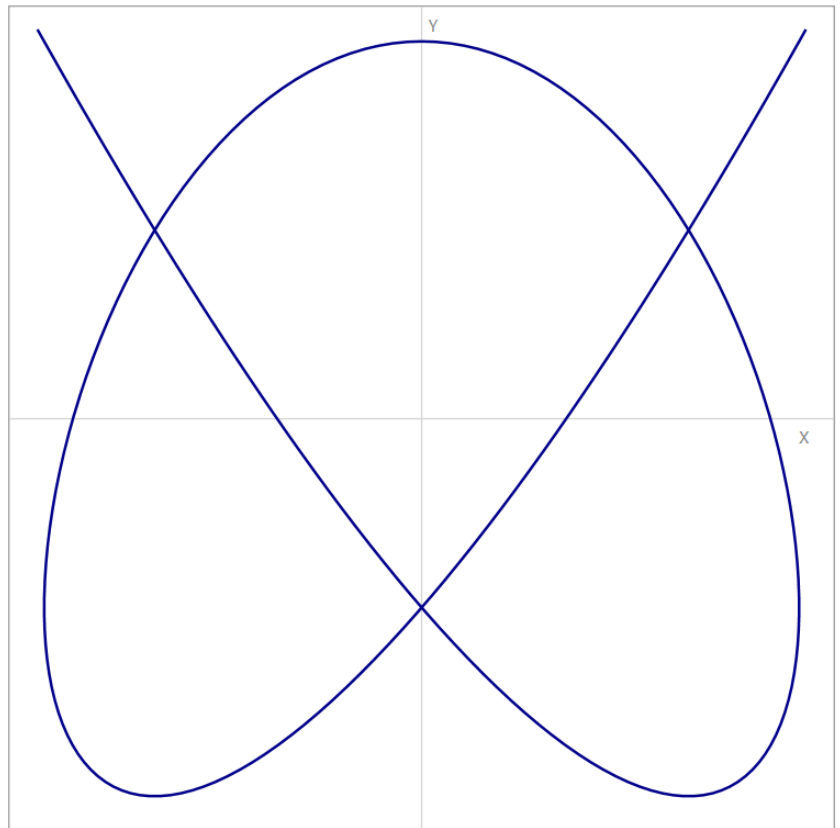
1 : 1

1 : 2

2 : 3

3 : 4

Побудовано успішно. Кількість точок: 835.



6. Робота інтерфейсу програми

Користувач вводить амплітуди A_x та A_y , частоти f_x та f_y , фази ϕ_x та ϕ_y у градусах і крок часу Δt . Для фази ϕ_y додатково передбачено Slider. Після натискання кнопки «Побудувати» команда BuildCommand запускає побудову.

У нижній частині панелі розміщені кнопки типових співвідношень частот 1:1, 1:2, 2:3 та 3:4. Після вибору співвідношення параметри частот оновлюються, а фігура перебудовується.

Повідомлення про помилки виводяться у вікні програми. Некоректні значення амплітуд, частот або кроку часу не повинні призводити до аварійного завершення програми.

7. Висновок

У ході лабораторної роботи було досліджено принцип побудови фігур Ліссажу як результату накладання двох гармонічних коливань. Було створено WPF-застосунок мовою C#, у якому параметри моделі задаються користувачем, а координати кривої розраховуються програмно та відображаються за допомогою Polyline.

Під час виконання роботи було використано прив'язку даних Binding, INotifyPropertyChanged та ICommand. Логіку математичних розрахунків винесено до окремого класу моделі, що відповідає вимогам розділення відповідальностей. Також реалізовано перевірку введення, підтримку різних співвідношень частот та однаковий масштаб координатних осей.